

Asincronismo, Promesas y Peticiones HTTP

1. Entendiendo el Asincronismo (Event Loop vs. Hilos)

- **En C#:** Se utiliza ejecución multihilo (Multithreading). Si se realiza una operación bloqueante en el hilo principal de la interfaz (UI Thread), la pantalla se congela.
- **En JavaScript:** El entorno ejecuta sobre un **único hilo principal** (Single-Threaded). Para evitar congelar la interfaz de la aplicación mientras se espera una respuesta del servidor, las operaciones de red y disco se ejecutan de forma **Asíncrona**.

2. De Task<T> en C# a Promise en JavaScript

En C#, para manejar operaciones asíncronas utilizaban Task o Task<T>. En JavaScript utilizamos el objeto **Promise (Promesa)**.

Una Promesa representa un valor que estará disponible inmediatamente, en el futuro o nunca. Atraviesa 3 estados:

1. **Pending (Pendiente):** Estado inicial, esperando la respuesta del servidor o proceso.
2. **Fulfilled / Resolved (Resuelta):** La operación finalizó con éxito y retornó el resultado.
3. **Rejected (Rechazada):** La operación falló (error de red, servidor fuera de línea, etc.).

3. Manejo Moderno de Asincronismo: async / await

La sintaxis de async/await en JavaScript es prácticamente idéntica a la de C#.

C# (.NET HttpClient):

```
public async Task ObtenerDatosAsync() {
    try {
        var client = new HttpClient();
        string respuesta = await
client.GetStringAsync("https://api.ejemplo.com/usuarios");
        Console.WriteLine(respuesta);
    }
    catch (Exception ex) {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

JavaScript (fetch API):

```
const obtenerUsuarios = async () => {
    try {
        // Petición HTTP GET asíncrona
        const respuesta = await fetch("https://api.ejemplo.com/usuarios");

        if (!respuesta.ok) {
            throw new Error(`Error en el servidor: ${respuesta.status}`);
        }

        // Deserialización del JSON
        const usuarios = await respuesta.json();
        console.log("Usuarios recibidos:", usuarios);
    }
}
```

```

    } catch (error) {
      console.error("Error al realizar la petición:", error.message);
    }
  };

  // Llamada a la función asíncrona
  obtenerUsuarios();

```

4. Guía Práctica de Verbos HTTP

El intercambio de información entre el Cliente y el Servidor sigue la arquitectura de servicios REST:

Verbo HTTP	Operación CRUD	Descripción
GET	Read (Leer)	Solicita información al servidor. No envía cuerpo de mensaje (body).
POST	Create (Crear)	Envía una nueva entidad en el cuerpo (body) de la petición para crearla.
PUT	Update (Actualizar)	Reemplaza o actualiza un registro completo existente en el servidor.
DELETE	Delete (Eliminar)	Remueve un registro específico del servidor identificándolo por ID.

5. Ejercicio Práctico Sugerido (Ejecutable con Node.js)

Consigna:

1. Crear un archivo llamado ejercicio.js en VS Code.
2. Crear una función asíncrona obtenerTarea(id).
3. Consumir el endpoint público de prueba <https://jsonplaceholder.typicode.com/todos/1> utilizando fetch y async/await.
4. Manejar los posibles errores mediante bloques try/catch e imprimir el título de la tarea en la consola.
5. Ejecutar el archivo desde la terminal de VS Code con el comando: node ejercicio.js.